

Neural network verification — cheat sheet

NNV-CS · REV A · Sept 2026

Vocabulary

Property — “for every input x in region R : $\varphi(x)$ ”. **Region** — the inputs the claim covers; a design decision. **Proof** — holds for every point. **Counterexample** — one concrete failing input, in your units. **Timeout** — an honest “too hard”. **Sound** — never claims a false property. **Complete** — always answers, given time. **Robustness** (ε -ball) — same class for every input within ε . **Embedding gap** — requirement in feet/mmHg vs solver in normalised tensors. **Hyperproperty** — about two evaluations (monotonicity, fairness): needs an encoding. **ONNX / VNN-LIB** — network / property exchange formats. **VNN-COMP** — annual competition (2025: 8 tools).

Vehicle syntax card

```
types & declarations

type Input  = Tensor Real [7]
type Output = Tensor Real [3]
low = 0      -- name an index
@network
f : Input -> Output
@parameter
eps : Real
@parameter(infer=True)
n : Nat
@dataset
xs : Vector Input n
@dataset
ys : Vector (Index 3) n
```

```
predicates & properties

valid : Input -> Bool
valid x = 0 ≤ x ! 0 ≤ 100 and x ! 5 = 0
```

```
advises : Input -> Index 3 -> Bool
advises x c = forall j . j ≠ c =>
  f x ! c > f x ! j
```

```
@property
p : Bool
p = forall x . valid x and x ! 1 ≤ 91
  => not (advises x low)
```

```
robustAround : Input -> Index 3 -> Bool
robustAround x c = forall d .
  (forall i . -eps ≤ d ! i ≤ eps)
  and valid (x + d) => advises (x + d) c
```

```
@property
q : Vector Bool n
q = foreach i . robustAround (xs ! i) (ys ! i)
```

Annotations (@network, @parameter, @dataset) go on their own line; put named indices *above* the network. Records (x.spo2) read well but do not combine with datasets in 0.27.1.

Command line

```
verify / typecheck / export

$ vehicle typecheck -s spec.vcl
$ vehicle verify -s spec.vcl \
  -n f:model.onnx -y p \
  -d xs:data.idx -d ys:labels.idx \
  -p eps:0.5 --solver Marabou \
  -a "--timeout=30" [-c cache/]
$ vehicle compile queries \
  --format VNNLibQueries -o out/ \
  -s spec.vcl -n f:model.onnx -e p
```

-y may repeat; omit it to verify all properties. --solver takes a name on PATH or a file path. Vehicle has no timeout of its own — always pass Marabou’s, and cap the wall clock too.

Reading results

result:  - Marabou proved no counterexample exists →

VERIFIED

result: x - Marabou found a counterexample then x: [...] →

COUNTEREXAMPLE

(units of the spec)

result: ? - Marabou timed out → **UNKNOWN**

Vector properties end with verified: a/n falsified: b/n timed-out: c/n.

Warning about strict inequalities: > is relaxed to $\geq$; ties on the boundary count as failures (and can appear as spurious counterexamples in fairness specs — add a margin).

Limits to remember

- Quantified variables only **linearly** (no $x \cdot y$).
- No **alternating** quantifiers (forall x . exists y).
- One** network application per query: hyperproperties need a twin (self-composed) network — and two *identical* copies can make the solver loop; prove those structurally.
- No Boolean inputs: pin flags with $\equiv$.
- if/or/max multiply queries; prefer relations.
- Marabou’s ONNX reader: Gemm/MatMul/Add/Relu/Reshape/... — no Concat, Slice, Div. Fold normalisation into layer 1.
- NP-complete: 16–32 hidden units → seconds; vision-scale → minutes to never.

What a good property looks like

- One sentence about every input in a named region, in the problem’s units.
- The region is plausible and bounded on every input.
- Thresholds sit away from where the training labels change (here: a full unit).
- You know what a counterexample would look like and who must see it.
- You have written down what the property does *not* cover.